

Project Design Phase-II

Solution Requirements (Functional & Non-functional)

Date	28 September 2025
Team ID	SWTID-2026-2056
Project Name	Implement Client Script & UI Policy (Incident)
Maximum Marks	4 Marks

Functional Requirements:

Following are the functional requirements of the proposed solution.

FR No.	Functional Requirement (Epic)	Sub Requirement (Story / Sub-Task)
FR-1	UI Policy – Field Behaviour Control	Make Close notes mandatory when Incident State is Resolved Hide Opened by field when State is Resolved Make Priority, Urgency and Severity read-only when State is Resolved Enable Reverse if false so attributes restore when State changes away from Resolved Execute UI Policy on form load and on condition change
FR-2	UI Policy – Conditional Visibility	Show additional fields (e.g., Impact assessment, Communication plan) when Major Incident flag is true Hide technical fields (Assignment group, Work notes) for End Users / Callers Control related-list visibility based on Incident State or category Support Global and view-specific UI Policies
FR-3	Client Script – onLoad	Set initial field attributes and default values when Incident form loads Display informational messages on form open based on current State or Priority Pre-populate fields (Location, Contact number) from Caller record on load of existing Incidents
FR-4	Client Script – onChange	Change Priority field background colour based on selected Priority value (Critical = red, etc.) Auto-populate Location and phone when Caller is selected Show field messages or alerts when State changes to Resolved or On Hold Dynamically show/hide or make fields mandatory based on Category / Subcategory selection Support access to oldValue for comparison logic
FR-5	Client Script – onSubmit Validation	Prevent form submission if Close notes is empty when State is Resolved Validate mandatory business rules client-side before save Display clear error messages and return false to block submit Allow successful submit only when all client-side validation rules pass
FR-6	Client Script – Server Interaction	Use GlideAjax or g_form.getReference() to fetch additional data from server Update form fields with data returned from Script Include Support asynchronous callbacks without blocking the UI
FR-7	Configuration & Administration	Create, activate and deactivate UI Policies on the Incident table Create onLoad, onChange, onSubmit and onCellEdit Client Scripts Set UI Type (Desktop / Mobile / All) for Client Scripts Control order of execution via Order field Add optional Execute if true / Execute if false scripts inside UI Policies
FR-8	Best-Practice Decision Logic	Prefer UI Policy for simple declarative field control (no scripting required) Use Client Script when complex logic, old-value access or onSubmit validation is needed Ensure UI Policies execute after Client Scripts; resolve conflicts in favour of UI Policy

Non-functional Requirements:

Following are the non-functional requirements of the proposed solution.

NFR No.	Non-Functional Requirement	Description
NFR-1	Usability	Client Scripts and UI Policies must provide immediate, clear visual feedback (field highlighting, mandatory indicators, field messages and alerts) so agents can complete Incident forms efficiently with minimal training. Error messages must be user-friendly and guide the agent to corrective action.
NFR-2	Security	Client-side logic can be bypassed; therefore critical data integrity and access rules must also be enforced server-side via ACLs, Data Policies and Business Rules. Scripts must not expose sensitive data or allow privilege escalation. UI Type and role-based conditions should restrict script execution appropriately.
NFR-3	Reliability	UI Policies and Client Scripts must execute consistently across form loads, field changes and submissions. Reverse-if-false behaviour of UI Policies must reliably restore previous field states. Scripts should handle null/empty values and unexpected data gracefully without throwing uncaught client-side errors.
NFR-4	Performance	Client-side scripts must execute quickly (typically < 100 ms for simple logic) so the form remains responsive. Avoid heavy synchronous GlideRecord queries; prefer GlideAjax with asynchronous callbacks. Minimise the number of simultaneous onChange scripts on frequently updated fields to prevent UI lag.
NFR-5	Availability	Because Client Scripts and UI Policies run in the browser, they remain available as long as the ServiceNow instance and the user's browser session are active. Scripts must degrade gracefully if a dependent Script Include is temporarily unavailable (e.g., show a non-blocking message instead of breaking the form).
NFR-6	Scalability	The solution must support a growing number of concurrent agents working on Incident forms without degradation. Prefer declarative UI Policies over complex Client Scripts where possible. Scripts should be scoped and ordered efficiently so that additional policies/scripts can be added later without significant performance impact.
NFR-7	Maintainability	All Client Scripts and UI Policies must follow naming conventions, contain clear short descriptions and inline comments. Complex logic should be modularised (e.g., reusable Script Includes). Documentation of when to use UI Policy versus Client Script must be maintained for future developers.
NFR-8	Compatibility	Scripts and policies must function correctly on both Desktop UI and Service Portal / Mobile (when UI Type = All). They should be tested across supported browsers and remain compatible with future ServiceNow platform upgrades (Zurich and later releases).